

Believe Writeup by 菜叶片

ID	661
作者	Gropers
系统	Linux
难度	Baby

初始信息收集

nmap端口扫描

```
A yepian 19:34 10GiB/26GiB
• sudo nmap -sS -A --min-rate 1000 -p- 192.168.56.12
[sudo] yepian 的密码 :
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-22 19:35 +0800
Nmap scan report for 192.168.56.12
Host is up (0.00015s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.3 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.67 ((Unix))
| http-cookie-flags:
|   /:
|   PHPSESSID:
|_    httponly flag not set
|_http-title: Login
|_http-server-header: Apache/2.4.67 (Unix)
2222/tcp  open  ssh      OpenSSH 10.3 (protocol 2.0)
54321/tcp open  ssh      OpenSSH 10.3 (protocol 2.0)
MAC Address: 08:00:27:AA:63:EB (Oracle VirtualBox virtual NIC)
```

端口扫描结果

端口号	服务	简要分析
22	SSH	N/A
80	Apache httpd 2.4.67	php编写的登录面板
2222	SSH	N/A
54321	SSH	N/A

攻击Web应用

收集信息

fuzz根目录 /FUZZ /FUZZ.php 探测到 /dashboard.php

此外没有其他页面

登录绕过

猜测 /index.php 密码校验 可能是...

- 数据库查询
- PHP字符串比较

尝试1: 测试数据库注入

往输入框传递SQLi NoSQLi XPath LDAP注入测试payload 均失败

尝试2: 测试PHP弱比较

传递数组或null 均失败

尝试3: 绕过/dashboard.php身份校验

尝试POST PUT DELETE等多个http方法 均失败

尝试4: 弱口令

试了几个常见的 均失败

看样子并没有办法绕过登录...

从SSH找到凭据

发现SSH的banner有凭据 登录web应用

最不可能有攻击向量的服务泄漏了敏感信息吗...有意思

进入仪表板 发现是一个web终端 发送了几个命令发现有过滤

貌似除了ls和cat都不能用 试了多种绕过方式都不行

cat只能读到yu.txt 从yu.txt得到提示

```
ssh just port 22 ? [emoji]
```

```
ls /home 得到用户名 yu
```

登录SSH

尝试凭据复用 登录SSH 通过54321端口登录成功

hint.txt中有提示

```
The walls have ears, and the logs have a heartbeat.  
it keeps whispering the truth. Just...
```

提示去看logs `find / -user yu -o -group yu 2>/dev/null` 扫全盘

注意到 `/var/log/.hidden/.ghost_radio`

这是个命名管道 看一下发现输出believe

`sudo -l` 发现可以以root身份运行/root/be 要求输入密码

输入believe通过认证 home目录下二进制be得到4655权限

利用后门get root

反编译二进制be

Decompile: main - (be)

```

14  int i;
15
16  encrypted[0] = 0x24;
17  encrypted[1] = 0x53;
18  encrypted[2] = 0x27;
19  encrypted[3] = 0x50;
20  encrypted[4] = 0x78;
21  encrypted[5] = 0x4b;
22  encrypted[6] = 0x9d;
23  encrypted[7] = 0xab;
24  encrypted[8] = 0xab;
25  encrypted[9] = '~';
26  encrypted[10] = 0xf0;
27  encrypted[0xb] = 0xa1;
28  encrypted[0xc] = 0x49;
29  encrypted[0xd] = 0xf2;
30  encrypted[0xe] = 0xea;
31  encrypted[0xf] = '~';
32  encrypted[0x10] = 0xec;
33  encrypted[0x11] = 0x6c;
34  encrypted[0x12] = 0x82;
35  encrypted[0x13] = 0x77;
36  generate_key(local_1c);
37  printf(&LoginRequired);
38  iVar1 = __isoc23_scanf("%255s",pInput);
39  if (iVar1 == 1) {
40      lenInput = strlen((char *)pInput);
41      lenInput' = (int)lenInput;
42      rc4_init(local_238,local_1c,7);
43      rc4_crypt(local_238,pInput,lenInput');
44      if (lenInput' == 0x14) {
45          local_10 = 1;
46          for (i = 0; i < 0x14; i = i + 1) {
47              if (pInput[i] != encrypted[i]) {
48                  local_10 = 0;
49                  break;
50              }
51          }
52          if (local_10 == 0) {
53              puts(&LoginFail);
54          }
55          else {
56              puts(&LoginSuccess);
57              get_shell();
58          }
59      }
60      else {
61          puts(&LoginFail);
62      }
63      uVar2 = 0;
64  }
65  else {
66      uVar2 = 1;
67  }
68  return uVar2;

```

```
69 }
70
```

generate_key函数生成rc4 key

rc4将输入值加密 密文与 24532750784b9dabab7ef0a149f2ea7eec6c8277 比较

继续看generate_key函数

```
2 void generate_key(long param_1)
3 {
4     byte OriKey [7];
5     int i;
6
7     OriKey [0] = 0xc8;
8     OriKey [1] = 0xcf;
9     OriKey [2] = 0xc6;
10    OriKey [3] = 0xc3;
11    OriKey [4] = 0xcf;
12    OriKey [5] = 0xdc;
13    OriKey [6] = 0xcf;
14    for (i = 0; i < 7; i = i + 1) {
15        *(byte *) (param_1 + i) = OriKey [i] ^ 0xaa;
16    }
17    *(undefined1 *) (param_1 + 7) = 0x0;
18    return;
19 }
```

c8cfc6c3cfdccf 与0xaa逐字节异或 最后再加个null byte终止

逆向解出passphrase

key异或解密出believe

passphrase解密出Believe_in_yourself!

输入passphrase得到root shell